

Day 2, exercise 2: build and deploy a triage agent

DRAFT. The prompts are still being tested.

Goal: by the end of the day, an agent reads every new tip, picks a desk, flags urgency and duplicates, and writes a short summary for the editor. You will have tested it before letting it run on its own, and you can see exactly what it did.

As on Day 1, **GitHub Copilot in Agent mode** writes the code. You decide what the agent is for, what it may do, and whether it is good enough to trust.

There are three parts, one after each topic of the day:

Part	Topic
1	What is an agent?
2	From prototype to agent
3	Deploying and operating it

The prompts below are suggestions. Rephrase them in your own words if you like, but keep what they ask for.

Before you start

- You completed exercise 1: your `AGENTS.md` has your own rules, including a `## Documentation` section. Step 9 tests it.
- You completed the Day 2 setup: you have the Day 2 files (including `app/llm.py`), and `.\.venv\Scripts\python -m app.check_model` reports that tool calling works.
- Your model key is saved in `C:\Users\<your name>\.tip-portal\key.txt`, outside the project. If Copilot ever asks to read that file, say no: it doesn't need the key to build and test the agent.
- You are on your Day 2 branch, and Copilot Chat is open (`Ctrl+Alt+I`).

Part 1 · Prompt, workflow or agent?

Step 1 · Triage by hand

Before building anything, try the simplest thing that could work: a prompt. In Copilot Chat, switch to **Ask** mode, attach `data/seed_tips.json` (paperclip icon, or drag the file into the chat) and ask:

For tips 1 to 7 in this file: which desk should handle it (politics, business, environment, health, local, or none), how urgent is it (high, normal, low), is it a duplicate of an earlier tip, and give a one-sentence summary for an editor. Answer as a table.

Look at the answer. It's probably quite good. Now discuss with your neighbour:

- Tomorrow 40 new tips arrive. **Who** runs this prompt, and **when**?
- Where does the answer go? Who copies it into the tip portal?
- To spot duplicates, it needed *all* the tips at once. What happens with 5,000 tips in the database?

- What did it get wrong, and how would you know if nobody checks?

Step 2 · Decide what needs an agent

	Who decides the steps?	Example in triage
Prompt	You, every time	Asking the chat to summarise one tip
Workflow	Your code, in a fixed order	For every new tip: summarise, pick a desk, save
Agent	The model, using tools you give it	Deciding <i>which</i> earlier tips to search for, to spot a duplicate

Pick a desk and write a summary: a fixed workflow could do that. Spotting a duplicate needs the model to decide what to look up. That's where an agent earns its extra cost and risk.

✅ **Done when** you can say which part of triage needs an agent, and why the rest doesn't.

Part 2 · From prototype to agent

Step 3 · Scope the job

Switch the chat to **Plan** mode (or start your prompt with "*Don't edit any files yet.*").

Plan a triage agent for the tip portal, in a new file `app/triage.py`.

Job: for each tip with status 'new', decide the desk, the urgency, whether it duplicates an earlier tip, and write a one-sentence summary for an editor. Save the result in the `triage` table and set the tip's status to 'triaged'. If the urgency is high, set the status to 'escalated' instead, so the editor on duty sees it straight away.

How: a tool-calling loop using `get_model()` from `app/llm.py`. Triage one tip per conversation. Give the model these tools:

- `get_tip(tip_id)`: the tip to triage
- `search_tips(query)`: earlier tips matching a word or phrase, to spot duplicates
- `save_triage(tip_id, desk, urgency, duplicate_of, summary)`: save the result

Stop after at most 10 model calls per tip. Log every model call, tool call and error in the `agent_log` table.

Add a command `flask triage` that triages all new tips, with a `--dry-run` option that prints the results without saving anything. Tests must use `FakeModel` and never call the real model. List the files you will change and the tests you will add.

Review the plan as if you were granting a new colleague access. Look for:

- **Actions:** what can the agent *change* in the database? Is that more than the job needs?
- **Data:** what does each tool return to the model? Open the plan and check.
- **Limits:** what stops it from looping forever, or triaging the same tip twice?
- **Failure:** what happens if the model gives an invalid desk, or calls a tool that doesn't exist?

Send corrections until the plan looks right.

✅ **Done when** you can name every action the agent can take, and you'd sign off on that list.

Step 4 · Build it against a fake model

Switch back to **Agent** mode.

Implement the plan. Write the tests first using FakeModel, then the code. Run the tests after each step.

While it works:

- **Read the approval prompts**, as on Day 1.
- Open one of the new tests. Can you see the scripted conversation the FakeModel plays back? Testing against a fake model means you can test the agent's *code* (does it save the right thing? does it stop?) without paying for, or depending on, the real model.

When the tests pass, commit:

```
git add -A
git commit -m "Triage agent with tests"
```

✅ **Done when** the tests pass and your work is committed.

Step 5 · Test it before you trust it

Passing tests prove the code works. They don't prove the *model* makes good decisions. For that, compare it with answers you agree with. `data/triage_expected.json` has the expected desk, urgency and duplicate for each sample tip (sometimes more than one answer is acceptable).

Add a command `flask triage-eval` that runs the agent in dry-run mode on all tips and compares the results with `data/triage_expected.json`. Print a table per tip (expected vs. actual, match or not) and the percentage correct for desk, urgency and duplicates. Add a test using FakeModel.

Then run it in the VS Code terminal:

```
.\.venv\Scripts\python -m flask --app app triage-eval
```

Discuss with your neighbour:

- Where does it disagree? Is the agent wrong, or is the expected answer wrong?
- Run it again. Do you get the same result?
- **Would you let it run unattended?** What score, on which field, would you need first? A wrong desk costs little. A missed urgent tip, or a wrong escalation, costs a lot.

✅ **Done when** you have an eval score and have decided what score you'd need before turning it on.

Part 3 · Deploying and operating it

Step 6 · Connect it to the portal

Show the triage result on the /editor page: desk, urgency, summary, and a link to the tip it duplicates. Tips that haven't been triaged yet show "Not triaged yet". Add a test.

Now run the agent for real:

```
.\.venv\Scripts\python -m flask --app app triage
```

Start the app and open /editor. Then go live: submit a **new** tip through the form (make it a duplicate of the harbour story, or something urgent), run `flask triage` again, and check that only the new tip was triaged.

Commit when it works.

✅ **Done when** a tip you submitted through the form shows up triaged on /editor.

Step 7 · Watch what it actually does

An agent you can't observe is an agent you can't trust.

Add a page /editor/log that shows the `agent_log`, newest run first: for each run, the tip, every model call (with tokens used) and every tool call with its arguments and result.

Open the page and look at one run closely:

- How many model calls did one tip take? How many tokens did the whole run use?
- Which tools did it call, and with what search words?
- **What data did the tools return to the model?** Was any of it more than triage needs?
- Did it ever call a tool with invalid arguments? What happened then?

✅ **Done when** you can explain, from the log alone, why the agent marked one tip as a duplicate.

Step 8 · Grounding: what it can see decides how good it is

The agent's decisions are only as good as what it has access to. Try it:

1. Ask Copilot to temporarily remove the `search_tips` tool from the agent.
2. Run the eval again:

```
.\.venv\Scripts\python -m flask --app app triage-eval
```

3. What happened to the duplicate score? Did it say "no duplicate", or did it **invent** one?
4. Undo the experiment:

```
git restore .
```

Discuss: the model was the same, the prompt was the same. What changed? What else could you give it access to that would make it better, and what would that cost in risk?

✅ **Done when** your agent is back to how it was at your last commit.

Step 9 · Document the agent

You built an agent that makes decisions about tips. Someone else, or you in a month, needs to know what it does and what it may touch. In exercise 1 you wrote a `## Documentation` section for your `AGENTS.md`. Now it gets its first real test.

Open `README.md`. The data model section still only describes the `tips` table.

Did your agent update the README while it built the triage agent, without being asked? If so, your rules work. If not, your rule is too weak or missing: ask now, and then fix the rule so it happens next time.

Document the triage agent in `README.md`: what it does, how to run it (`flask triage`, `--dry-run`, `flask triage-eval`), the tools it can call and what each one can change, its limits, and the `triage` and `agent_log` tables. Read the code and `app/schema.sql` to get the facts. Don't guess.

Then **read what it wrote against the code**. Agents write documentation that sounds right:

- Do the commands, tool names and table columns match what is really there?
- Does it say what the agent **can change**? Would a new colleague know what it is allowed to do?
- Does it contain anything it shouldn't: a key, or the contents of a real tip?
- Is there a diagram, if your rule asks for one? Does it match the flow in the log from step 7?

Fix what is wrong, and commit:

```
git add -A
git commit -m "Document the triage agent"
```

✅ **Done when** a colleague could run the triage agent, and know what it can change, from the `README` alone.

Finished early?

- **Run it on a schedule:** add `flask triage --watch` that checks for new tips every minute. What should happen if the model service is down?
- **Budget:** add a limit on the total tokens per run, and stop with a clear message when it is reached.
- **Bad answers:** what does your agent do if the model returns a desk that doesn't exist? Write a test with `FakeModel` that proves it.
- Ask: *"What should be in `AGENTS.md` about the triage agent that isn't there yet?"*

Before you leave

```
git add -A
git commit -m "Day 2 done"
```

Bring the same laptop on Day 3. On Day 3 you'll try to trick your own agent into doing something it shouldn't, so keep it exactly as it is.